

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
“ЛЭТИ” ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**ОТЧЕТ
по лабораторно-практической работе № 2
«СОЗДАНИЕ WEB-ПРОЕКТА С ИСПОЛЬЗОВАНИЕМ
СИСТЕМЫ СБОРКИ MAVEN»
по дисциплине «Web-программирование»**

Выполнил: Барченков П.А.

Факультет: КТИ

Группа: №3312

Подпись преподавателя: _____

Санкт-Петербург

2025

Содержание

Цель работы	3
Порядок выполнения	3
Ход работы.....	3
Вывод.....	6
Приложение	6

Цель работы

Знакомство с основными понятиями сборки и средствами создания web-проекта с помощью Maven.

Порядок выполнения

1. Создать Maven-проект с архетипом maven-archetype-webapp 1.4.
2. Выполнить сборку Web-приложения, используя команду Maven install.
3. Скопировать war-файл проекта в каталог Web-сервера Apache Tomcat.
4. Открыть и просмотреть содержание war-файла.
5. Запустить Web-приложение в браузере.

Ход работы

Создадим в IntelliJ IDEA новый Maven-проект со следующими параметрами (рисунок 1):

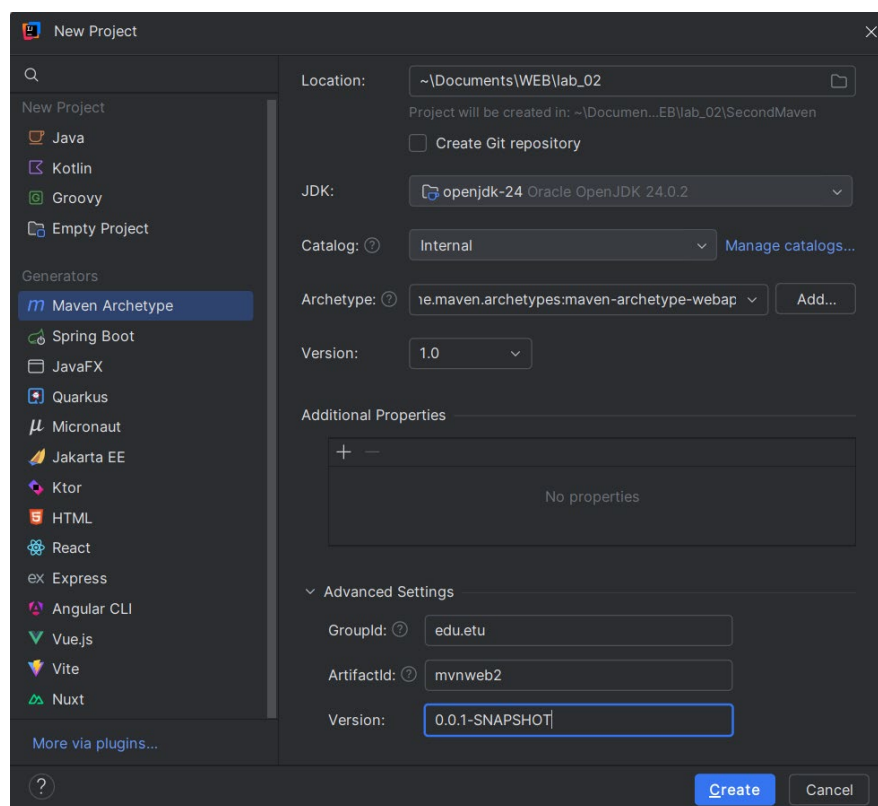


Рисунок 1 – Окно создания Maven-проекта

После откроется основное окно среды разработки (рисунок 2).

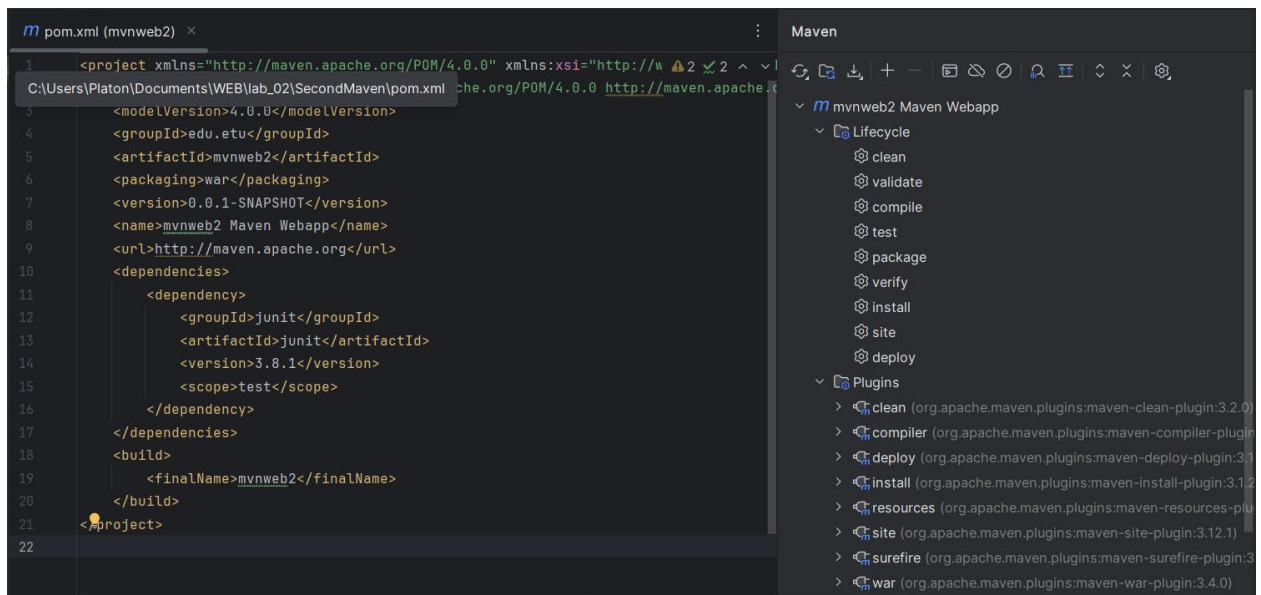


Рисунок 2 – Основное окно IntelliJ IDEA

Очистим проект с помощью команды `clean`. Далее выполним команду `install` (рис. 3). В структуре проекта появилась папка `target`.

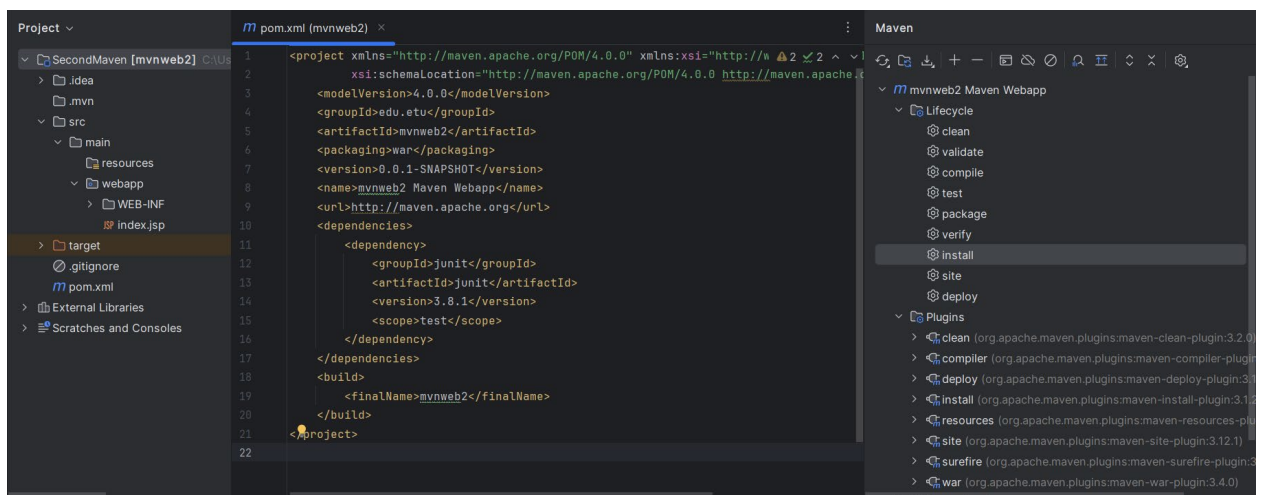


Рисунок 3 – Результат выполнения команды «install»

В папке `target` находится архив `mvnweb2.war`. Изучим его содержимое с помощью программы `7zip` (рис. 4):

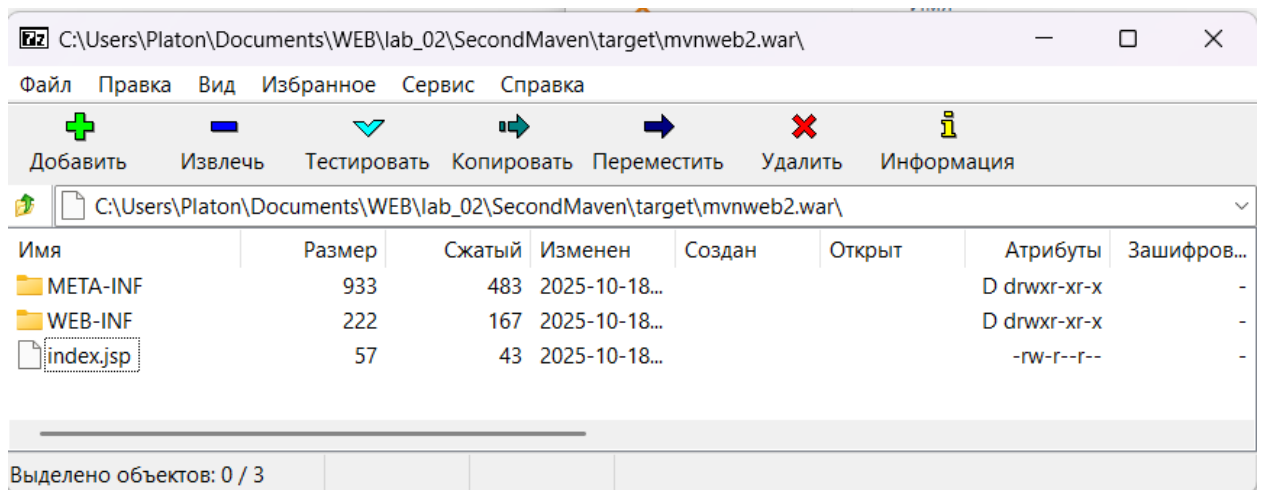


Рисунок 4 – Содержимое mvnweb2.war

Внутренняя структура архива mvnweb2.war (рис. 5):

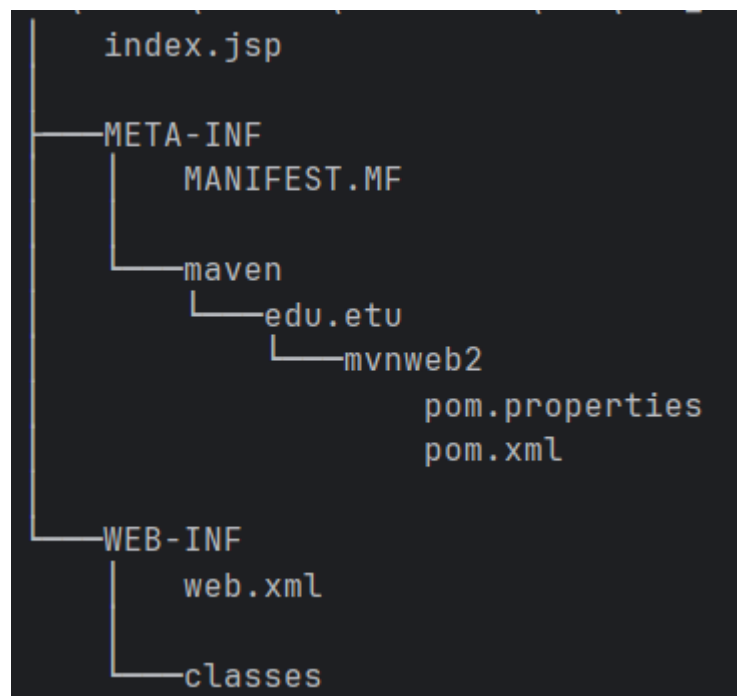


Рисунок 5 –mvnweb2.war

Положим mvnweb2.war в каталог apache-tomcat-10.1.46\webapps. После чего выполним startup.bat. Проверим что по адресу <http://localhost:8080/mvnweb2/> (рисунок 6) работает наше приложение.

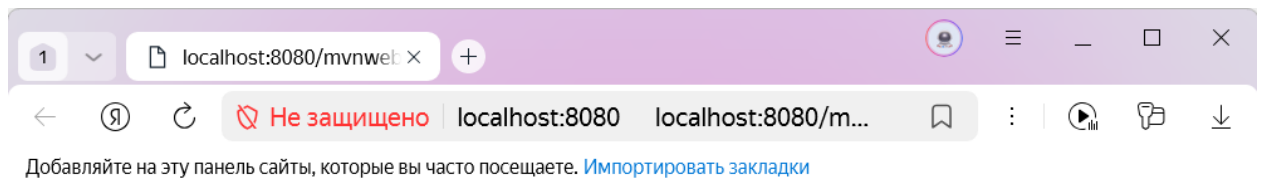


Рисунок 6 – Экранная форма Web-приложения

Вывод

В ходе работы был создан и собран Maven-проект *mvnweb2*, получен файл *mvnweb2.war*, который успешно развернут на сервере Apache Tomcat 10.1.47. Сервер корректно запустился на порту 8080, приложение доступно в браузере по адресу <http://localhost:8080/mvnweb2/>. Работоспособность проекта подтверждена отображением страницы *index.jsp*.

Приложение

Код pom.xml

```
<!-- POM: конфигурация Maven-проекта -->
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

    <!-- Модель POM -->
    <modelVersion>4.0.0</modelVersion>

    <!-- Координаты артефакта: кто и что собираем -->
    <groupId>edu.etu</groupId>          <!-- Группа/орг. -->
    <artifactId>mvnweb2</artifactId>    <!-- Имя приложения -->
    <version>0.0.1-SNAPSHOT</version>  <!-- Версия (SNAPSHOT = в разработке) -->

    <!-- Тип сборки: WAR для веб-приложения -->
    <packaging>war</packaging>

    <name>mvnweb2 Maven Webapp</name>
    <url>http://maven.apache.org</url>
```

```

<!-- Свойства проекта: версия Java для компиляции -->
<properties>
  <!-- release сообщает компилятору, под какую платформу компилировать.
    17 — надёжный выбор; можно поставить 21, если у вас JDK 21+ и
    плагины поддерживают -->
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<!-- Зависимости, нужные на этапе компиляции/тестов -->
<dependencies>

  <!-- Jakarta Servlet API (Tomcat 10+).
    scope=provided: контейнер (Tomcat) подставит эту библиотеку во
    время выполнения,
    а в WAR мы её не кладём, чтобы не было конфликтов версий. -->
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.0.0</version>
    <scope>provided</scope>
  </dependency>

  <!-- JUnit 4 для модульных тестов (запускается плагином surefire на
  фазе test).
    scope=test: используется только при тестировании. -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>

</dependencies>

<build>
  <!-- Имя итогового артефакта: target/mvnweb2.war -->
  <finalName>mvnweb2</finalName>

  <!-- Плагины Maven: выполняют фазы жизненного цикла сборки -->
  <plugins>

    <!-- maven-clean-plugin: очищает target/ на фазе clean -->
    <plugin>
      <artifactId>maven-clean-plugin</artifactId>
      <version>3.3.2</version>
      <!-- goal clean связывается по умолчанию с фазой clean -->
    </plugin>

    <!-- maven-resources-plugin: копирует ресурсы (webapp,
    *.properties) на фазах process-resources/testResources -->
    <plugin>
      <artifactId>maven-resources-plugin</artifactId>
      <version>3.3.1</version>
    </plugin>

    <!-- maven-compiler-plugin: компилирует Java-код на фазе
    compile/testCompile -->
    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
      <configuration>
        <!-- Использует свойство maven.compiler.release -->

```

```

        <release>${maven.compiler.release}</release>
    </configuration>
</plugin>

test -->
    <!-- maven-surefire-plugin: запускает модульные тесты на фазе
test -->
    <plugin>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
        <!-- По умолчанию подхватывает классы *Test.java -->
    </plugin>

package -->
    <!-- maven-war-plugin: упаковывает веб-приложение в WAR на фазе
package -->
    <plugin>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
        <configuration>
            <!-- web.xml берётся из src/main/webapp/WEB-INF/web.xml;
            можно явно указать, если лежит в другом месте -->
            <failOnMissingWebXml>false</failOnMissingWebXml>
        </configuration>
    </plugin>

    <!-- maven-install-plugin: кладёт собранный артефакт в локальный
репозиторий ~/.m2 на фазе install -->
    <plugin>
        <artifactId>maven-install-plugin</artifactId>
        <version>3.1.2</version>
    </plugin>

    <!-- maven-deploy-plugin: (опционально) отправляет артефакт в
удалённый репозиторий на фазе deploy -->
    <plugin>
        <artifactId>maven-deploy-plugin</artifactId>
        <version>3.1.1</version>
        <!-- Для ЛР обычно не нужен; оставлен для полноты -->
    </plugin>

    <!-- (опционально) javadoc: генерирует документацию по коду по
команде mvn javadoc:javadoc -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-javadoc-plugin</artifactId>
        <version>3.6.3</version>
    </plugin>

</plugins>
</build>
</project>

```